# Dev Pratap Singh

+91-8260851668 | devpratap9089@gmail.com | dev-pratap-singh-a694b6366 | 9089huckleberry

## Technical Skills

**Core Stack**: C#, .NET Framework (academic/personal projects), React.js, JavaScript, OOP, SOLID Principles, Git
**Languages**: C++, C, C#, Python, SQL, JavaScript, Bash Scripting
**Software Engineering**: Data Structures & Algorithms, Design Patterns, Software Architecture, Agile/Scrum exposure (academic team projects), Code Review, Unit Testing (GTest)
**Systems & Networking**: Operating Systems, DBMS, Computer Networks, Multithreading, Distributed Systems, TCP/IP, REST APIs, Node.js, Express.js, Socket.IO
**Build, Test & Dev Tools**: CMake, Make, GTest, Docker, Git, Postman, Ollama, Linux
**Data & Databases**: NumPy, Pandas, MySQL, MongoDB
**Certifications**: Google Cloud (Data Warehousing with BigQuery, ML APIs, Predictive Modeling via BigQuery ML)

## Education

**National Institute of Technology Tiruchirapalli** **Trichy, Tamil Nadu**
*Bachelor of Technology in Computer Science and Engineering* ***(CGPA: 7.7/10)*** *November 2022 – May 2026*

## Experience

**PwC India** **Bangalore, Karnataka**
*Data Analytics and Cloud Intern* *May 2025 – July 2025*

- Improved internal data accessibility by **42%** by **architecting** a centralized enterprise data warehouse in **Google BigQuery**, consolidating **2.5M+** supply chain records across distributed schemas using structured, modular data-contract design.
- Eliminated **18+ hours** of weekly manual preprocessing by **automating** end-to-end ETL pipelines in **Google Dataprep**, enforcing schema-validation rules across departments through reusable, testable transformation logic.
- Reduced forecasting error (MAPE) by **26%** by **engineering** time-series predictive models in **BigQuery ML**, applying iterative validation cycles to improve model reliability for inventory planning.
- Boosted cross-functional operational efficiency by **31%** by **deploying** real-time executive dashboards on **GCP**, collaborating directly with business stakeholders to translate multi-source analysis into actionable decisions.

## Projects

**High-Concurrency Server Engine (500+ Clients, Sub-12ms Latency)** | GitHub **April 2024 – May 2024**
*Technologies: C++, POSIX Sockets, Multithreading, CMake, GTest*

- Sustained sub-**12ms** message latency across active connections by **engineering** a high-throughput, multi-client TCP routing architecture in C++ using low-level POSIX sockets.
- Reduced thread-context switching overhead by **35%** by **designing** an asynchronous server with a thread-pool pattern, applying **SOLID-aligned** modular separation between connection, routing, and synchronization logic to support **500+ concurrent sessions**.
- Eliminated data races under high-frequency workloads by **implementing** fine-grained synchronization with C++17 mutexes and condition variables, achieving thread-safe broadcasting.
- Reached **85% branch coverage** by **building** a **test-driven** GTest suite of **30+ unit and integration cases** within a reproducible CMake pipeline, validating concurrency-critical code paths before integration.

**PacketScope: Real-Time Network Traffic Analyzer** | GitHub **March 2025 – May 2025**
*Technologies: C++, Raw Sockets, TCP/IP, Libpcap, CMake*

- Accelerated packet decode cycles by **42%** by **implementing** zero-copy memory parsing modules to extract protocol metadata, endpoint pairs, and payload lengths.
- Enabled real-time diagnostics at **15,000 packets/second** by **engineering** an in-memory sliding-window packet buffer for live network analysis.
- Alerted administrators to simulated anomalies within **1.5 seconds** by **building** automated traffic-analysis pipelines that calculate live bandwidth utilization statistics.

**VectorRAG: Distributed Vector Search Engine (14x Retrieval Speedup)** | GitHub **June 2025 – August 2025**
*Technologies: C++, HNSW, RAG, Ollama, REST APIs, CMake*

- Achieved a **14x speedup** in semantic retrieval over **768-dimensional embeddings** by **architecting** a low-latency vector database in C++ implementing HNSW graph layers, a KD-Tree, and brute-force indexing.
- Improved semantic context retrieval for localized knowledge bases by **engineering** a local Retrieval-Augmented Generation pipeline integrating Ollama (Llama 3.2) and nomic-embed-text.
- Enabled vector insertion, bulk indexing, and search benchmarking by **exposing** database operations through **10+ RESTful API endpoints**.
- Supported crash recovery and durable storage by **implementing** a write-ahead log (WAL) and on-disk persistence layer across server restarts.